

Roadmap Technique Backend Django REST Framework

Guide de Maîtrise : Transactions Mobile Money, Automation Celery & Système de Notifications

Projet : Centralisation Concours & Annales (Backend API)

Auteur : Feyegnong Mbakoup Purentin (Fyg)

Stack Cible : Django, DRF, PostgreSQL, Celery, Redis

Date : Août 2026

1. Notions Clés par Module Fonctionnel

A. Module Paiement Mobile Money (MTN / Orange Money)

- **Webhooks & Vues DRF** : Réception des callbacks asynchrones des API Orange/MTN. Traitement sécurisé des requêtes POST entrantes sans authentification JWT classique (vérification de signature HMAC/Header secret).
- **Transactions Atomiques (`django.db.transaction.atomic`)** : Isolation complète de l'opération bancaire pour éviter les incohérences de données en cas d'interruption serveur.
- **Gestion de l'Idempotence** : Prévention des doubles crédits en vérifiant l'unicité de la référence de transaction externe.

B. Module Filtrage Dynamique & Recherche

- **`django-filter` Integration** : Construction de filtres déclaratifs complexes sur les sessions de concours (filière, niveau, statut, intervalle de dates).
- **Optimisation des Requêtes ORM** : Utilisation systématique de `select_related()` et `prefetch_related()` pour éliminer le problème des requêtes SQL $N+1$.
- **Préférences Utilisateur** : Utilisation de champs `JSONField` ou de relations Many-To-Many pour associer les filtres sauvegardés au profil de l'étudiant.

C. Module Automation & Task Scheduling (Celery + Redis)

- **Celery Workers** : Exécution en arrière-plan des tâches lourdes (génération de reçu PDF, envoi de mails/SMS) sans bloquer les réponses de l'API REST.
- **Celery Beat** : Planificateur de tâches pour la mise à jour automatique de l'état des concours (ex: annulation/clôture automatique chaque minuit).
- **Django Signals (`post_save`)** : Déclenchement automatique des événements lorsqu'un administrateur modifie une épreuve ou un concours.

D. Module Système de Notification Multi-Canal

- **Modélisation In-App Notification** : Stockage des alertes utilisateur (Lu/Non lu, Titre, Lien, Timestamp).
- **Push Notifications & SMS** : Intégration du SDK Firebase Cloud Messaging (FCM) via une tâche asynchrone Celery.
- **Django Channels (Optionnel)** : WebSockets pour la diffusion instantanée de notifications Push sur le client SvelteKit/Mobile.

2. Récapitulatif du Stack Technique & Bibliothèques

Fonctionnalité	Package / Outil Django	Rôle Technique
API Core	<code>django-rest-framework</code>	Architecture des endpoints, Serializers, ViewSets.
Sécurité Auth	<code>django-rest-framework-simplejwt</code>	Gestion des tokens d'accès JWT et rafraîchissement.
Filtrage	<code>django-filter</code>	Filtres d'API déclaratifs et dynamiques.
Broker / Cache	<code>Redis</code>	Base de mémoire vive pour le cache et les files Celery.
Tâches Async	<code>celery</code> + <code>django-celery-beat</code>	Traitement asynchrone et planificateur chron.
Mobile Push	<code>firebase-admin</code>	Envoi de notifications vers l'app mobile Flutter/React Native.

3. Plan d'Apprentissage & Méthodologie Proactive

- Phase 1 - Modélisation & Core REST** : Structurer la base de données PostgreSQL (Utilisateurs, Concours, Transactions). Mettre en place `SimpleJWT` et le filtrage personnalisé.
 - Phase 2 - Sandbox Paiement Mobile** : Créer une vue Webhook de test. Implémenter `transaction.atomic()` et valider le paiement simulé.
 - Phase 3 - Configuration Celery + Redis** : Lancer Redis sur Docker/Natif. Créer une tâche Celery Beat qui clôture automatiquement les concours dont la date d'échéance est atteinte.
 - Phase 4 - Pipeline de Notifications** : Connecter un Signal Django `post_save` sur le modèle Concours pour déclencher l'envoi asynchrone de notifications en cas d'annulation ou modification.